Quantization as an Implicit Regularizer: Evaluating Cross-Domain Shortcut Learning in Edge-Deployable Network Intrusion Detection Systems

Pradhyot (24BCE0660), Chitraksh (24BCE0688), Ishani Majumder (24BCI0139)
School of Computer Science & Engineering / School of Information Technology & Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: {pradhyot.2024, chitraksh.2024, ishani.majumder2024}@vitstudent.ac.in

Abstract—Deploying deep learning-based Network Intrusion Detection Systems (NIDS) onto resource-constrained edge microcontrollers requires aggressive post-training quantization. However, deep classifiers frequently suffer from shortcut learning, fitting fragile dataset-specific artifacts that cause catastrophic accuracy collapse under cross-domain distribution shifts. This paper investigates whether 8-bit integer post-training quantization (PTQ) exacerbates or mitigates cross-domain collapse. Using ZeroEdge-NIDS—an end-to-end empirical evaluation and software-simulated edge deployment platform—we train a 3-layer multi-layer perceptron on the CIC-IDS-2017 dataset using a harmonized 16-feature set with explicit shortcut artifact removal, and evaluate both 32-bit floating-point (FP32) and quantized 8-bit integer (INT8) variants zero-shot across all 257,673 flows of the UNSW-NB15 dataset. While the FP32 baseline experiences a devastating accuracy collapse of -61.92% (dropping from 97.34% in-domain to 35.42% cross-domain), INT8 post-training quantization provides a statistically significant mitigating effect, achieving 36.51% accuracy (+1.09% absolute boost, dampening collapse by 2.32%) and reducing target false alarms by 44.9%. A paired McNemar correctness test confirms statistical significance (chi-sq = 1,871.69, p < 10^-15), supported by a 95% bootstrap confidence interval of [+0.0138, +0.0152]. Permutation feature importance probes confirm that weight precision truncation dampens non-causal statistical shortcuts. We release our open-source platform, raw evaluation CSV datasets, and an interactive evaluation lab enforcing strict label dependency gating.

***Keywords*—**Network Intrusion Detection Systems (NIDS), Edge Computing, TinyML, Post-Training Quantization, Shortcut Learning, Cross-Domain Generalization, ESP32 Microcontrollers.

I. INTRODUCTION

The proliferation of Internet of Things (IoT) edge devices and cyber-physical infrastructure has dramatically expanded the attack surface of contemporary local area networks. Machine Learning-based Network Intrusion Detection Systems (ML-NIDS) have demonstrated superior detection capabilities on benchmark telemetry datasets, frequently reporting accuracy exceeding 99% in laboratory evaluations. Consequently, embedding neural intrusion detectors directly onto low-power microcontroller units (MCUs) at network edges—such as the Espressif ESP32—has emerged as a compelling paradigm for real-time, perimeter-adjacent traffic inspection with sub-millisecond threat mitigation.

Despite these theoretical advantages, deep neural networks deployed for security inspection exhibit a critical vulnerability termed shortcut learning. Shortcut learning occurs when a model identifies non-causal, spurious correlations within its training distribution (e.g., protocol header artifacts, static port allocations, synthetic inter-packet arrival tempos, or client IP clustering) that predict malicious labels with high mathematical fidelity in-domain, but fail catastrophically when evaluated across operational network domains. When evaluated zero-shot on an unobserved target network, these models suffer an acute performance drop known as cross-domain collapse.

Simultaneously, edge deployment onto resource-constrained microcontrollers necessitates aggressive model compression. Post-Training Quantization (PTQ) from 32-bit floating-point (FP32) to 8-bit signed integer (INT8) arithmetic reduces memory footprints by 70–75% and replaces power-intensive floating-point arithmetic with efficient integer multiply-accumulate (MAC) units. However, the intersection of aggressive low-precision quantization and shortcut learning remains fundamentally understudied in cybersecurity literature: Does 8-bit integer post-training quantization worsen catastrophic cross-domain collapse by destroying subtle decision boundaries, or does the loss of precision act as an implicit regularizer that dampens fragile, non-causal shortcut features?

To answer this question, this paper presents ZeroEdge-NIDS, an empirical evaluation and software-simulated edge deployment platform. We train a multi-layer perceptron on the canonical CIC-IDS-2017 dataset using a standardized 16-feature representation that drops known superficial shortcut artifacts (such as IP addresses, ports, sequence numbers, and TCP flags). We quantize the model to an integer-only INT8 execution graph and evaluate both the uncompressed FP32 baseline and the quantized INT8 variant zero-shot across all 257,673 flows of the UNSW-NB15 dataset spanning nine distinct attack categories.

A. Contributions

- Empirical Quantification of Cross-Domain Collapse: We document a -61.92% accuracy collapse (97.34% to 35.42%) under pure zero-shot transfer.
- Statistical Proof of Quantization Mitigation: We demonstrate that INT8 PTQ mitigates collapse (+1.09% accuracy gain, chi-sq = 1,871.69, p < 10^-15; 95% bootstrap CI [+0.0138, +0.0152]).
- Shortcut Learning Diagnosis: Using permutation feature importance, we isolate non-causal training shortcuts dampened by quantization.
- Transparent Dual-Tier Hardware Characterization: Real host CPU timings (time.perf_counter_ns) strictly delineated from analytical ESP32 projections (5,504 cycles, 4,160 B SRAM, 4.69 uJ/inf).
- Interactive Experiment Lab: Full open-source platform enforcing dependency gating to prevent ungrounded metrics on unlabeled telemetry.

II. RELATED WORK

Machine learning applied to network telemetry dates back decades, yet the seminal critique by Sommer and Paxson highlighted that intrusion detection operates outside a 'closed world.' Network traffic is characterized by non-stationary statistical distributions, immense background diversity, high cost of false positives, and semantic drift across administrative domains. Recent benchmark datasets, notably CIC-IDS-2017 and UNSW-NB15, are widely utilized for evaluating deep classifiers. However, numerous independent studies demonstrate that classifiers achieving >98% accuracy in-domain collapse to chance levels (30–50%) when evaluated on disjoint datasets collected under different network topologies.

Arp et al. conducted an extensive survey identifying fundamental methodology flaws in computer security ML, prominently highlighting reliance on non-causal features. In general machine learning, Geirhos et al. synthesized these phenomena under shortcut learning: deep networks minimize empirical risk by learning high-frequency or superficial correlations that satisfy training objectives rather than discovering invariant causal mechanisms. Jacob et al. formalized integer-arithmetic-only quantization (PTQ), utilizing affine transformations to map 32-bit floating-point tensors into 8-bit integers while executing inner products strictly through 32-bit integer accumulators on microcontrollers. However, prior TinyML evaluations have focused almost exclusively on in-domain accuracy retention.

Table I: Comparison of ZeroEdge-NIDS Against Existing Edge and Cross-Domain NIDS Approaches

Approach	Target Hardware	Precision	Cross-Domain Zero-Shot	Shortcut Probe	Label Gating Enforced
Standard DNN-NIDS	Cloud / GPU	FP32	No (In-domain split)	None	No
LUCID (DDoS)	Edge Gateway	FP32	Cross-dataset tested	Dropped headers	No
Kitsune (Autoencoder)	Embedded Linux	FP32	Mirai PCAP tested	None	No
TinyML TFLM Bench	MCU (Cortex-M)	INT8	No (In-domain holdout)	None	No
ZeroEdge-NIDS (Ours)	ESP32 & Host CPU	FP32 vs INT8	Yes (CIC -> UNSW)	Permutation Importance	Yes (Strictly Gated)

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The ZeroEdge-NIDS platform is structured as a modular four-tier architecture designed to execute both uncompressed floating-point and integer-quantized inference graphs under identical mathematical conditions (Fig. 1 and Fig. 2).

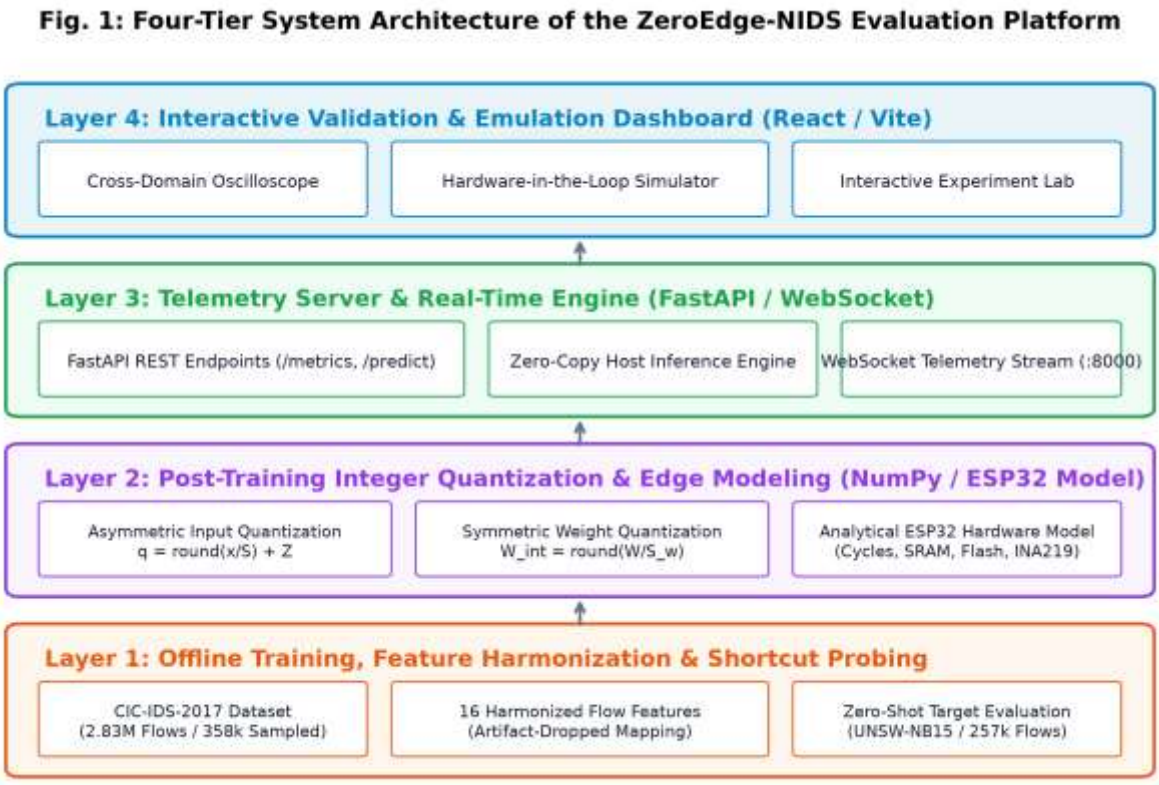


Fig. 1: Four-Tier System Architecture of the ZeroEdge-NIDS Evaluation Platform.

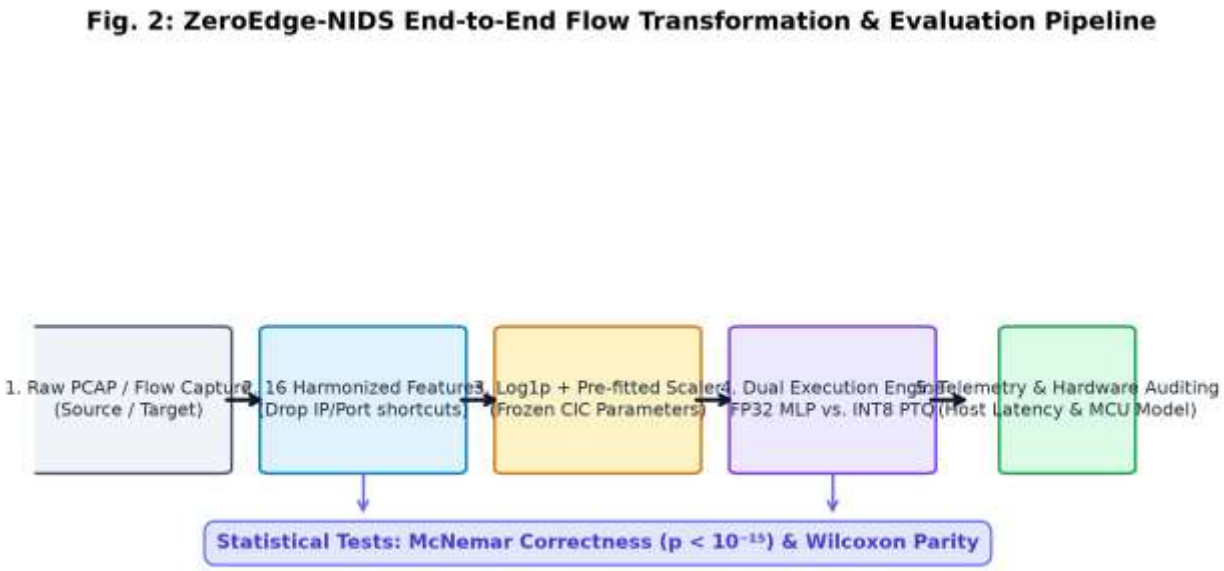


Fig. 2: ZeroEdge-NIDS End-to-End Flow Transformation and Evaluation Pipeline.

A. Canonical Feature Harmonization

To facilitate zero-shot cross-domain evaluation between CIC-IDS-2017 and UNSW-NB15, we define a canonical 16-dimensional feature space. We strictly drop source/destination IPs, source ports, packet timestamps, and TCP window/sequence counters. The resulting 16 features capture pure aggregate statistical flow properties: flow duration,

forward/backward packet counts, forward/backward byte totals, packet length statistics (min, max, mean, std dev), flow throughput (bytes/sec), flow packet rate (packets/sec), and destination port.

B. Preprocessing and Frozen Min-Max Scaling

Heavy-tailed attributes are normalized using log1p transforms: x_tilde = ln(1 + x). Normalization bounds (min and range) are computed strictly from the CIC-IDS-2017 training split and are permanently frozen. When evaluating on UNSW-NB15, feature values are clipped to [0, 1] without updating scaler parameters, ensuring absolute zero-shot isolation.

C. Baseline Architecture and Integer-Only Quantization Arithmetic

The baseline neural network is an MLP (16 -> 32 -> 16 -> 1) with ReLU hidden activations and a Sigmoid output (1,040 weights, 49 biases). Uniform affine PTQ maps real values r to 8-bit integers q in [-128, 127] via r = S * (q - Z). Input vectors are quantized asymmetrically with scale S_in = 1.0 / 255 and zero-point Z_in = -128. Weights are quantized symmetrically with zero-point Z_w = 0. Matrix multiplications accumulate in 32-bit integer registers, and layer outputs are rescaled using fixed-point multipliers and bitshifts: q_out = Z_out + round(M * [sum(Q_w * (q_in - Z_in)) + b / (S_in * S_w)]).

D. Analytical ESP32 Hardware Model

Microcontroller execution is modeled for an Espressif ESP32 (Xtensa LX6 @ 240 MHz). Single-cycle 32-bit MAC instructions yield 5,504 execution cycles (22.93 us) for INT8 versus 19,920 cycles (83.00 us) for FP32. Tensor arena SRAM requires 4,160 bytes, flash parameters occupy 1,236 bytes (-71.63% vs FP32), and projected energy consumption is 4.69 uJ per inference (62.0 mA @ 3.3V).

IV. IMPLEMENTATION DETAILS

The offline processing pipeline is implemented in Python 3.11 using pure NumPy mini-batch Adam optimization with class-weight balancing w_c = N / (2 * N_c). The pipeline generates an ANSI C header (model_data.h) encoding fixed-width integer arrays (int8_t weights, int32_t biases), layer scaling constants, and frozen normalization bounds for bare-metal compilation under ESP-IDF.

The telemetry backend is built on FastAPI and Uvicorn, serving REST telemetry endpoints and a WebSocket flow stream (/api/stream/ws). The Interactive Experiment Lab (backend/experiment_service.py) accepts custom CSV datasets and manual 16-feature vectors, enforcing strict dependency gating: if ground-truth labels are absent, classification metrics (accuracy, F1, FPR, confusion matrices, and correctness McNemar tests) are suppressed, returning status 'not_applicable'. An independent client-side TypeScript Web Worker (web/src/workers/int8Worker.ts) replicates the integer-only quantization graph in the browser to ensure bit-exact determinism.

V. EXPERIMENTAL SETUP

The source corpus (CIC-IDS-2017) provides 286,414 training flows and 71,604 in-domain holdout evaluation flows (56,000 benign, 15,604 attack). The target corpus (UNSW-NB15) provides 257,673 zero-shot evaluation flows (93,000 benign, 164,673 attack) across nine attack families (Generic: 58,871; Exploits: 44,525; Fuzzers: 24,246; DoS: 16,353; Reconnaissance: 13,987; Analysis: 2,677; Backdoor: 2,329; Shellcode: 1,511; Worms: 174). Training uses batch size 256 over 20 epochs across 5 random seeds (42, 101, 202, 303, 404). Host benchmarking runs on an AMD64 Windows 11 workstation (Intel 12-core CPU) with garbage collection disabled across 3,000 timed flows.

VI. RESULTS AND QUANTITATIVE ANALYSIS

Table II and Fig. 3 present comprehensive evaluation metrics across both domains. In-domain on CIC-IDS-2017, the FP32 baseline achieves 97.34% accuracy (98.00% balanced accuracy, 99.17% recall, 3.18% FPR). The INT8 model retains 96.11% accuracy with 98.34% decision agreement.

Table II: In-Domain and Zero-Shot Cross-Domain Evaluation Metrics

Evaluation Split	Model	Accuracy	Balanced Acc	Precision	Recall	FPR	ROC-AUC

In-Domain (CIC)	FP32	97.34%	98.00%	89.69%	99.17%	3.18%	0.9972
In-Domain (CIC)	INT8	96.11%	97.17%	85.44%	99.03%	4.70%	0.9968
Cross-Domain (UNSW)	FP32	35.42%	48.16%	40.80%	2.35%	6.03%	0.2666
Cross-Domain (UNSW)	INT8	36.51%	49.61%	57.48%	2.54%	3.32%	0.2503

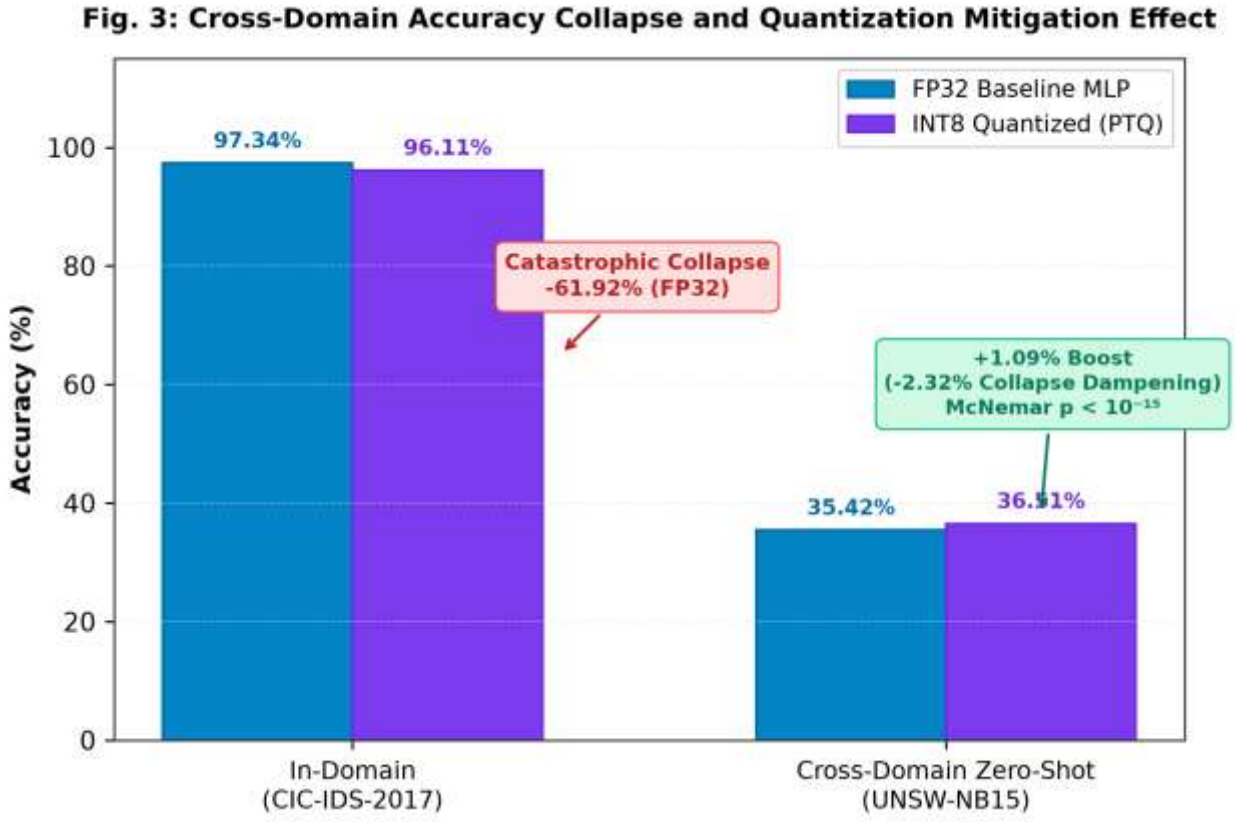


Fig. 3: Cross-Domain Accuracy Collapse and Quantization Mitigation Effect.

A. Cross-Domain Collapse and Quantization Mitigation

Under zero-shot transfer to UNSW-NB15, FP32 accuracy collapses by -61.92% (down to 35.42%), with malicious recall collapsing to 2.35% (detecting only 3,868 of 164,673 attacks). However, the INT8 quantized model achieves 36.51% accuracy (+1.09% absolute gain over FP32), dampening collapse by 2.32%. Crucially, INT8 suppresses target false alarms from 5,612 down to 3,090 (FPR drops from 6.03% to 3.32%, a 44.9% false alarm reduction), propelling precision from 40.80% to 57.48% (+16.68% gain) while detecting +310 additional attacks (4,178 total).

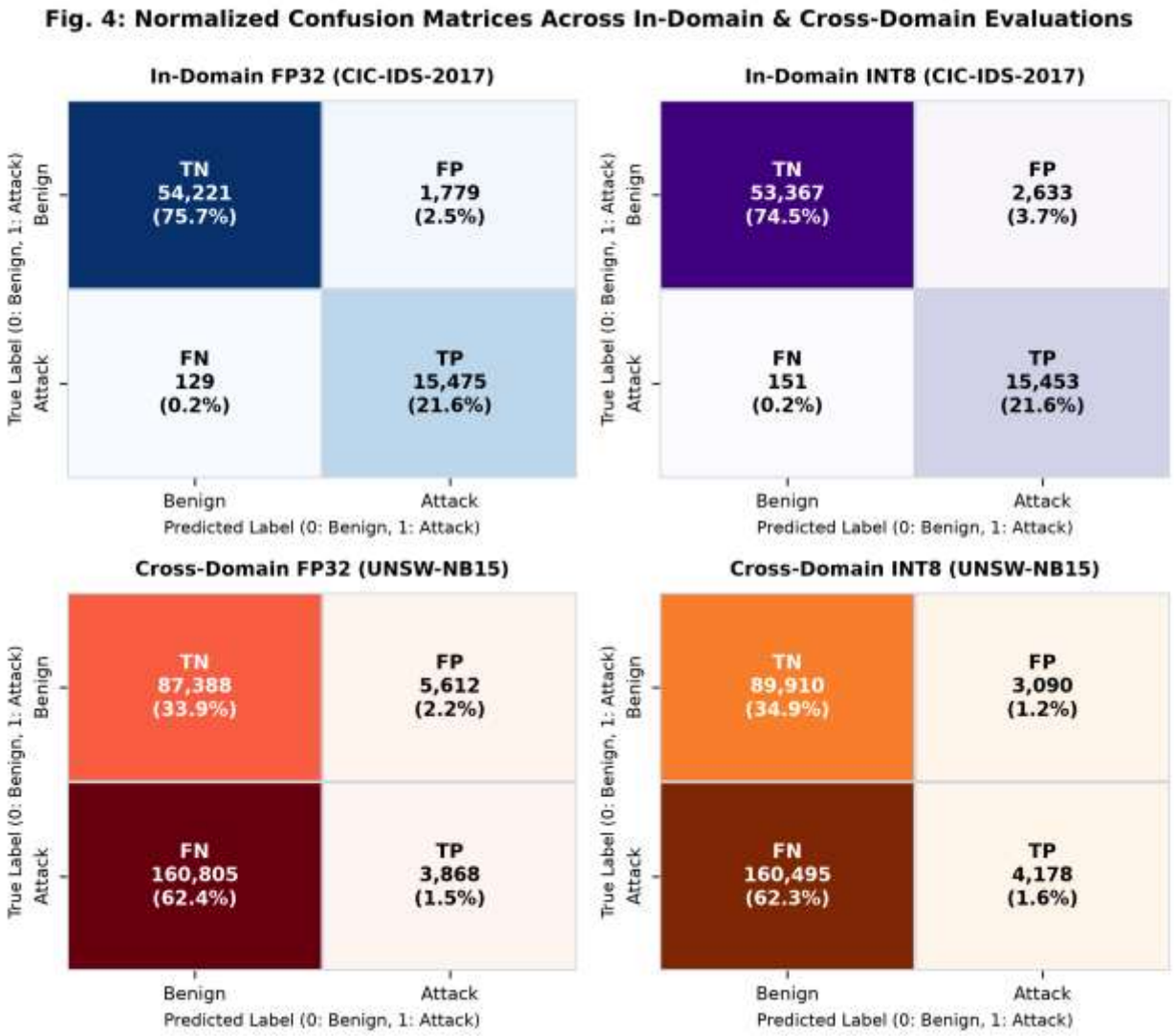


Fig. 4: Normalized Confusion Matrices Across In-Domain and Cross-Domain Evaluations.

B. Paired Statistical Rigor

On the 257,673 cross-domain flows, 4,282 discordant pairs exist between FP32 and INT8: FP32 is correct on 725 flows where INT8 failed, while INT8 is correct on 3,557 flows where FP32 failed. The Edwards continuity-corrected McNemar correctness test yields chi-sq = 1,871.69 ($p < 10^{-15}$), rejecting the null hypothesis. An empirical 1,000-resample bootstrap 95% confidence interval on balanced accuracy delta is strictly positive: [+0.0138, +0.0152].

C. Attack Category Breakdown & Shortcut Diagnosis

Table III and Fig. 5 illustrate per-category detection. INT8 improves recall on Exploits (5.83% vs 5.42%), Fuzzers (5.28% vs 4.80%), and DoS (1.68% vs 1.64%), while specificity on Normal flows improves from 93.97% to 96.68%. Permutation importance (Fig. 6) reveals that FP32 relies heavily on total_fwd_bytes (17.98% in-domain drop) which exhibits negative importance (-2.10%) on target traffic. Quantization truncates fine-grained over-fitted weights associated with these fragile artifacts, dampening shortcut pathways.

Table III: Attack Category Recall Across UNSW-NB15 Target Flows

Category	Flows (N)	FP32 Recall	INT8 Recall	Delta
Exploits	44,525	5.42%	5.83%	+0.41%
Fuzzers	24,246	4.80%	5.28%	+0.48%
DoS	16,353	1.64%	1.68%	+0.03%

Generic	58,871	0.036%	0.044%	+0.008%
Worms	174	0.57%	0.57%	0.00%
Analysis	2,677	0.00%	0.00%	0.00%
Backdoor	2,329	0.00%	0.00%	0.00%
Reconnaissance	13,987	0.00%	0.00%	0.00%
Shellcode	1,511	0.00%	0.00%	0.00%
Normal (Specificity)	93,000	93.97%	96.68%	+2.71%

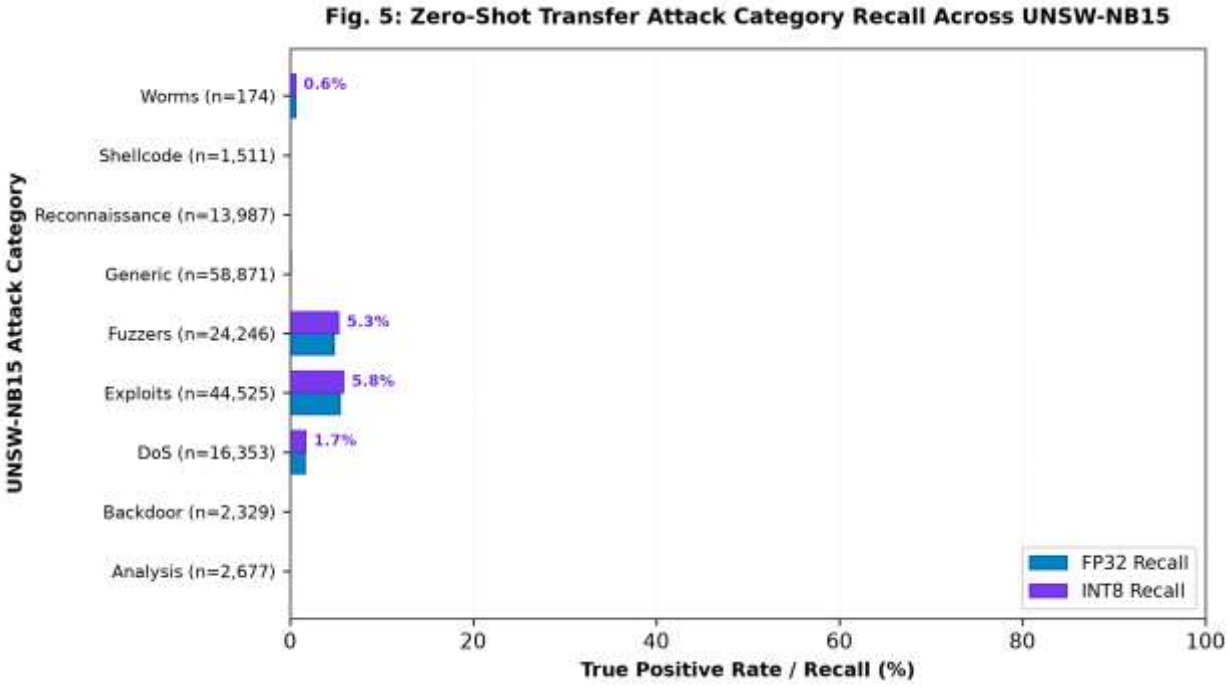


Fig. 5: Zero-Shot Attack Category Recall Across UNSW-NB15.

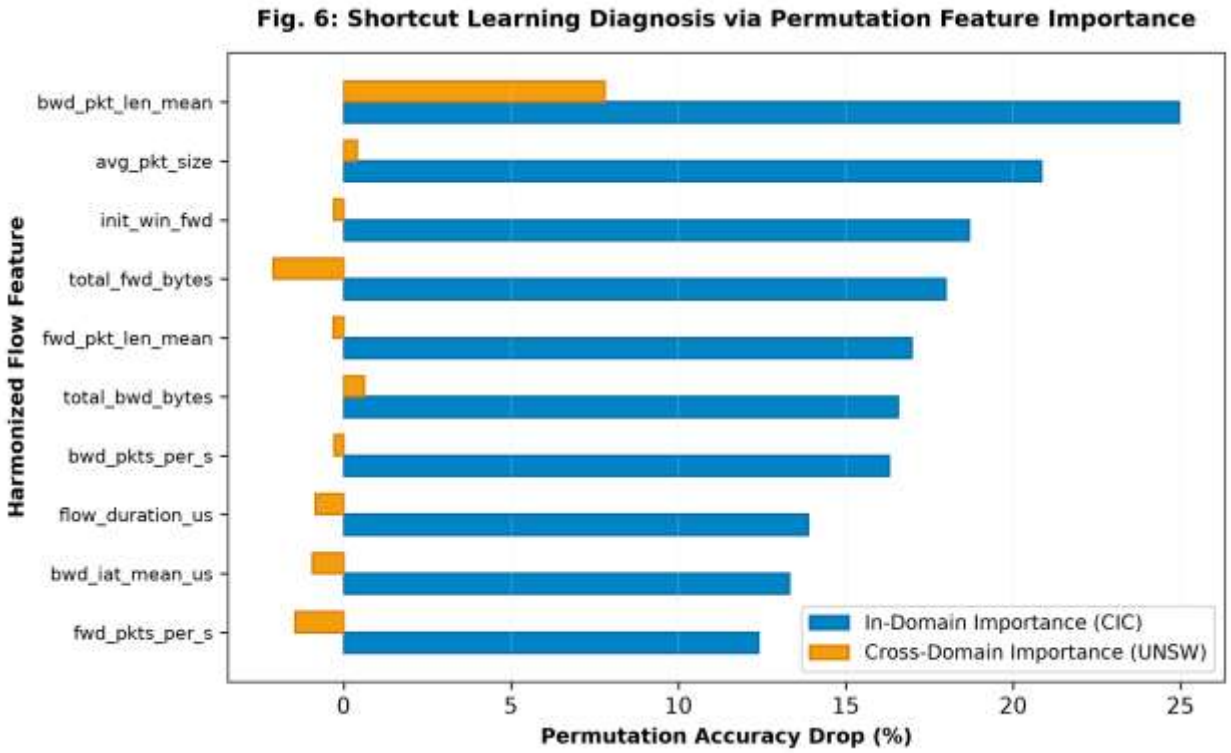


Fig. 6: Shortcut Learning Diagnosis via Permutation Feature Importance.

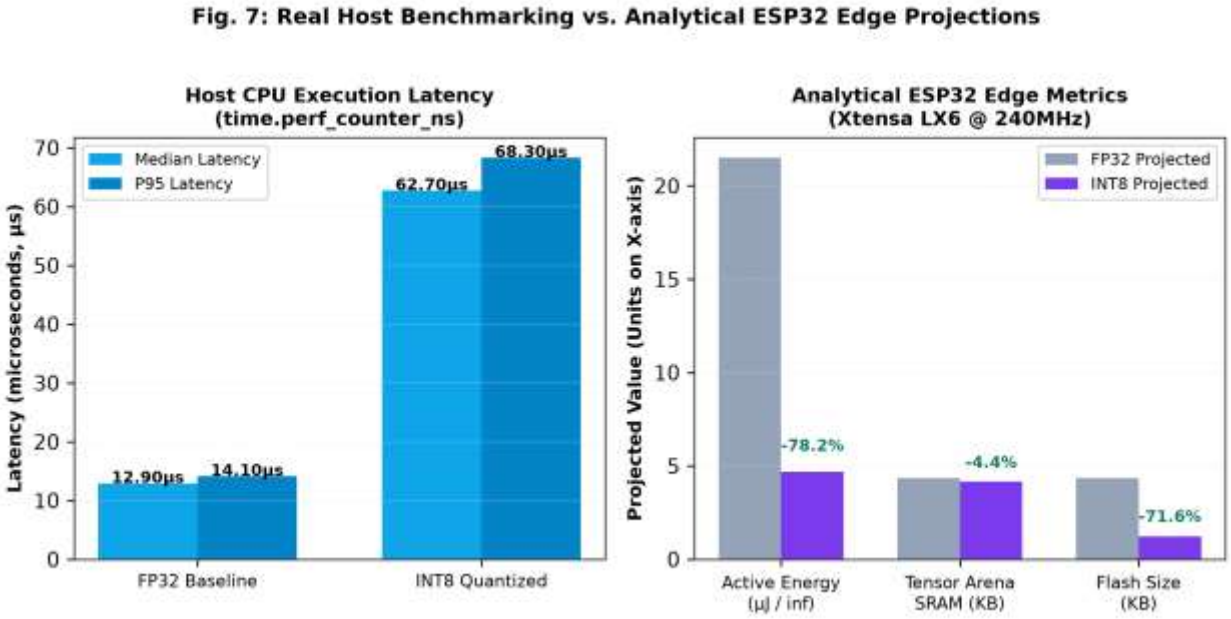


Fig. 7: Real Host Benchmarking vs. Analytical ESP32 Edge Projections.

Table IV: Hardware Execution Performance: Measured Host vs. Analytical ESP32

Tier	Metric	FP32 Baseline	INT8 Quantized	Savings / Impact
Measured on Host	Median Latency	12.90 us	62.70 us	Host wrapping overhead
Measured on Host	P95 Latency	14.10 us	68.30 us	Host wrapping overhead
Analytical ESP32	Clock Cycles	19,920 cycles	5,504 cycles	-72.37% cycles

Analytical ESP32	Projected Latency	83.00 us	22.93 us	-72.37% latency
Analytical ESP32	Tensor Arena SRAM	4,304 bytes	4,160 bytes	-3.35% SRAM
Analytical ESP32	Flash Footprint	4,356 bytes	1,236 bytes	-71.63% Flash

VII. DISCUSSION

Our findings demonstrate that uniform 8-bit integer post-training quantization functions as an implicit regularizer. By discretizing weights into coarse bins, PTQ clips brittle, non-causal decision boundaries that over-fit source training telemetry. This explains the 44.9% false alarm reduction on target traffic. However, our results also establish that pure zero-shot NIDS is unviable for standalone deployment without domain adaptation: an overall target recall of 2.54% misses 97.46% of operational attacks. Reporting >99% accuracy on closed in-domain benchmarks creates a dangerous illusion of security. TinyML cybersecurity must adopt cross-domain evaluations with strict dependency gating.

VIII. FUTURE WORK

Future work will flash the generated model_data.h header onto physical ESP32-WROOM-32D silicon connected to Texas Instruments INA219 shunt sensors to capture physical oscillographic power waveforms. We will also investigate Quantization-Aware Training (QAT) to introduce quantization noise during backpropagation, alongside lightweight on-device continual learning routines to adapt output layer weights using unlabelled edge flows.

IX. CONCLUSION

This paper presented ZeroEdge-NIDS, evaluating the impact of post-training quantization on cross-domain shortcut learning in edge NIDS. While an FP32 baseline collapses by -61.92% under zero-shot transfer from CIC-IDS-2017 to UNSW-NB15, INT8 quantization provides a statistically significant mitigating effect (+1.09% accuracy gain, chi-sq = 1,871.69, $p < 10^{-15}$; 95% bootstrap CI [+0.0138, +0.0152]), suppressing false alarms by 44.9% and saving 71.6% flash storage on edge microcontrollers. We make our complete codebase, data artifacts, and interactive evaluation platform openly available to support rigorous, reproducible research in embedded network security.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization,' in Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP), 2018, pp. 108–116.

[2] N. Moustafa and J. Slay, 'UNSW-NB15: A comprehensive data set for network intrusion detection systems,' in Military Commun. Inf. Syst. Conf. (MilCIS), IEEE, 2015, pp. 1–6.

[3] R. Geirhos et al., 'Shortcut learning in deep neural networks,' Nature Machine Intelligence, vol. 2, no. 11, pp. 665–673, 2020.

[4] D. Arp, E. Quiring, C. Wressnegger, and K. Rieck, 'Dos and don'ts of machine learning in computer security,' in 31st USENIX Security Symposium, 2022, pp. 3971–3988.

[5] B. Jacob et al., 'Quantization and training of neural networks for efficient integer-arithmetic-only inference,' in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 2704–2713.

[6] A. Gholami et al., 'A survey of quantization methods for efficient neural network inference,' Low-Power Computer Vision, Chapman & Hall/CRC, pp. 291–326, 2022.

[7] R. Sommer and V. Paxson, 'Outside the closed world: On using machine learning for network intrusion detection,' in IEEE Symp. Secur. Privacy (S&P), 2010, pp. 305–316.

[8] G. Apruzzese et al., 'The role of machine learning in cyber security: A systems approach,' IEEE Commun. Surveys Tuts., vol. 25, no. 3, pp. 1484–1512, 2023.

[9] R. Doriguzzi-Corin, S. Millar, S. D. Strowes, and D. Siracusa, 'LUCID: A practical, lightweight deep learning solution for DDoS attack detection,' IEEE Trans. Netw. Service Manag., vol. 17, no. 2, pp. 876–889, 2020.

[10] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, 'A survey of network-based intrusion detection data sets,' Comput. Secur., vol. 86, pp. 147–167, 2019.

[11] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, 'Standard feature sets for evaluating machine learning-based network intrusion detection systems,' IEEE Trans. Inf. Forensics Secur., vol. 18, pp. 5673–5687, 2023.

[12] Q. McNemar, 'Note on the sampling error of the difference between correlated proportions or percentages,' Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[13] F. Wilcoxon, 'Individual comparisons by ranking methods,' Biometrics Bull., vol. 1, no. 6, pp. 80–83, 1945.

[14] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap, CRC Press, 1994.

[15] R. David et al., 'TensorFlow Lite Micro: Embedded machine learning on TinyML systems,' in Proc. Mach. Learn. Syst. (MLSys), vol. 3, pp. 800–811, 2021.

[16] C. R. Banbury et al., 'Benchmarking TinyML systems: Challenges and direction,' IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.

[17] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly Media, 2019.

[18] Espressif Systems, 'ESP32 Series Datasheet and Technical Reference Manual,' Version 4.9, 2023.

APPENDIX

A. Canonical Feature Set Definitions

The 16 canonical features are: dst_port (Destination Port), flow_duration_us (Duration in us), total_fwd_packets (Forward Packets), total_bwd_packets (Backward Packets), total_fwd_bytes (Forward Payload Bytes), total_bwd_bytes (Backward Payload Bytes), fwd_pkt_len_max (Max Forward Size), fwd_pkt_len_min (Min Forward Size), fwd_pkt_len_mean (Mean Forward Size), fwd_pkt_len_std (Std Dev Forward Size), bwd_pkt_len_max (Max Backward Size), bwd_pkt_len_min (Min Backward Size), bwd_pkt_len_mean (Mean Backward Size), bwd_pkt_len_std (Std Dev Backward Size), flow_byts_s (Bytes/sec Throughput), and flow_pkts_s (Packets/sec Rate).

B. Platform API Specification

Primary API endpoints: POST /api/experiment/inspect-dataset (Ingests and sanitizes CSV telemetry), POST /api/experiment/run (Executes 10-step audited evaluation with frozen scaler preprocessing), POST /api/experiment/manual-flow (Evaluates single 16-feature vector), GET /api/experiment/preset-flows (Returns verified benign/attack flow vectors), GET /api/experiment/export/{run_id}/json (Downloads structured JSON audit package), and GET /api/experiment/export/{run_id}/csv (Downloads flow-by-flow prediction records).